

ACTS1200数据采集卡

驱动程序使用手册

北京阿尔泰科技发展有限公司

V6.00.00



前言

版权归北京阿尔泰科技发展有限公司所有，未经许可，不得以机械、电子或其它任何方式进行复制。
本公司保留对此文档更改的权利，方案后续相关变更时，请联系技术人员确认指标。

免责说明

订购产品前，请向厂家或经销商详细了解产品性能是否符合您的需求。

正确的运输、储存、组装、装配、安装、调试、操作和维护是产品安全、正常运行的前提。本公司对于任何因安装、使用不当而导致的直接、间接、有意或无意的损坏及隐患概不负责。

安全使用小常识

- 1.在使用产品前，请务必仔细阅读产品使用手册；
- 2.对未准备安装使用的产品，应做好防静电保护工作(最好放置在防静电保护袋中，不要将其取出)；
- 3.在拿出产品前，应将手先置于接地金属物体上，以释放身体及手中的静电，并佩戴静电手套和手环，要养成只触及其边缘部分的习惯；
- 4.为避免人体被电击或产品被损坏，在每次对产品进行拔插或重新配置时，须断电；
- 5.在需对产品进行搬动前，务必先拔掉电源；
- 6.对整机产品，需增加/减少板卡时，务必断电；
- 7.当您需连接或拔除任何设备前，须确定所有的电源线事先已被拔掉；
- 8.为避免频繁开关机对产品造成不必要的损伤，关机后，应至少等待 30 秒后再开机。

目 录

■ 1 版权信息与命名约定.....	2
1.1 版权信息.....	2
1.2 命名约定.....	2
■ 2 使用纲要.....	3
2.1 使用上层用户函数，高效、简单.....	3
2.2 如何管理设备.....	3
2.3 如何用连续采集方式取得 AD 数据.....	3
2.4 如何用有限采集方式取得 AD 数据.....	4
2.5 哪些函数对您不是必须的.....	6
■ 3 ACTS 设备操作函数接口介绍.....	7
3.1 设备驱动接口函数总列表（每个函数省略了前缀“ACTS1200_”）.....	7
3.2 设备对象管理函数原型说明.....	8
3.3 AI 模拟量输入实现函数原型说明.....	10
3.4 AI 硬件参数保存与读取函数原型说明.....	4
■ 4 硬件参数结构.....	6
4.1 AI 工作参数结构体（ACTS1200_AI_PARAM）.....	6
4.2 AI_CH_PARAM(AI 通道参数结构体).....	7
4.3 AI 状态参数结构（ACTS1200_AI_STATUS）.....	9
4.4 AI 主要信息结构（ACTS1200_AI_MAIN_INFO）.....	10
4.5 AI 采样范围信息结构体(ACTS1200_AI_SAMP_RANGE_INFO).....	11
4.6 AI 速率信息结构体(ACTS1200_AI_SAMP_RATE_INFO).....	11
5.1 AI 原码 LSB 数据转换成电压值的换算方法.....	12
5.2 AI 采集函数的 ADBuffer 缓冲区中的数据排放规则.....	12
■ 6 上层用户函数接口应用实例.....	13
6.1 简易程序演示说明.....	13
6.2 高级程序演示说明.....	13

1 版权信息与命名约定

1.1 版权信息

本软件产品及相关套件均属北京阿尔泰科技发展有限公司所有，其产权受国家法律绝对保护，除非本公司书面允许，其他公司、单位、我公司授权的代理商及个人不得非法使用和拷贝，否则将受到国家法律的严厉制裁。您若需要我公司产品及相关信息请及时与当地代理商联系或直接与我们联系，我们将热情接待。

1.2 命名约定

一、为简化文字内容，突出重点，本文中提到的函数名通常为基本功能名部分，其前缀设备名如 PCIexxxx_则被省略。如 ACTS1200_CreateDevice 则写为 CreateDevice。

二、函数名及参数中各种关键字缩写规则

缩写	全称	汉语意思	缩写	全称	汉语意思
Dev	Device	设备	DI	Digital Input	数字量输入
Pro	Program	程序	DO	Digital Output	数字量输出
Int	Interrupt	中断	CNT	Counter	计数器
Dma	Direct Memory Access	直接内存存取	DA	Digital convert to Analog	数模转换
AD	Analog convert to Digital	模数转换	DI	Differential	(双端或差分) 注：在常量选项中
Npt	Not Empty	非空	SE	Single end	单端
Para	Parameter	参数	DIR	Direction	方向
SRC	Source	源	ATR	Analog Trigger	模拟量触发
TRIG	Trigger	触发	DTR	Digital Trigger	数字量触发
CLK	Clock	时钟	Cur	Current	当前的
GND	Ground	地	OPT	Operate	操作
Lgc	Logical	逻辑的	ID	Identifier	标识
Phys	Physical	物理的			

以上规则不局限于该产品。

■ 2 使用纲要

2.1 使用上层用户函数，高效、简单

如果您只关心通道及频率等基本参数，而不必了解复杂的硬件知识和控制细节，那么我们强烈建议您使用上层用户函数，它们就是几个简单的形如 Win32 API 的函数，具有相当的灵活性、可靠性和高效性。诸如 InitDeviceAD、StartDeviceAD 等。而底层用户函数如 WriteRegisterULong、ReadRegisterULong、WritePortByte、ReadPortByte……则是满足了解硬件知识和控制细节、且又需要特殊复杂控制的用户。但不管怎样，我们强烈建议您使用上层函数（在这些函数中，您见不到任何设备地址、寄存器端口、中断号等物理信息，其复杂的控制细节完全封装在上层用户函数中。）对于上层用户函数的使用，您基本上不必参考硬件说明书，除非您需要知道板上插座等管脚分配情况。

2.2 如何管理设备

由于我们的驱动程序采用面向对象编程，所以要使用设备的一切功能，则必须首先用 CreateDevice 函数创建一个设备对象句柄 hDevice，有了这个句柄，您就拥有了对该设备的绝对控制权。然后将此句柄作为参数传递给相应的驱动函数，如 InitDeviceAD 可以使用 hDevice 句柄以程序查询方式初始化设备的 AD 部件，ReadDeviceAD-函数可以用 hDevice 句柄实现对 AD 数据的采样读取等。最后可以通过 ReleaseDevice 将 hDevice 释放掉。

2.3 如何用连续采集方式取得 AD 数据

当您有了 hDevice 设备对象句柄后，便可用 InitDeviceAD 函数初始化 AD 部件，关于采样通道、频率等参数的设置是由这个函数的 pADPara 参数结构体决定的。您只需要对这个 pADPara 参数结构体的各个成员简单赋值即可实现所有硬件参数和设备状态的初始化。然后用 StartDeviceAD 即可启动 AD 部件，开始 AD 采样，然后便可用 ReadDeviceAD 反复读取 AD 数据以实现连续不间断采样。当您需要暂停设备时，执行 StopDeviceAD，当您需要关闭 AD 设备时，ReleaseDeviceAD 便可帮您实现（但设备对象 hDevice 依然存在）。（注：ReadDeviceAD 虽然主要面对批量读取、高速连续采集而设计，但亦可用它以单点或几点的方式读取 AD 数据，以满足慢速、高实时性采集需要）。具体执行流程请看下面的图。

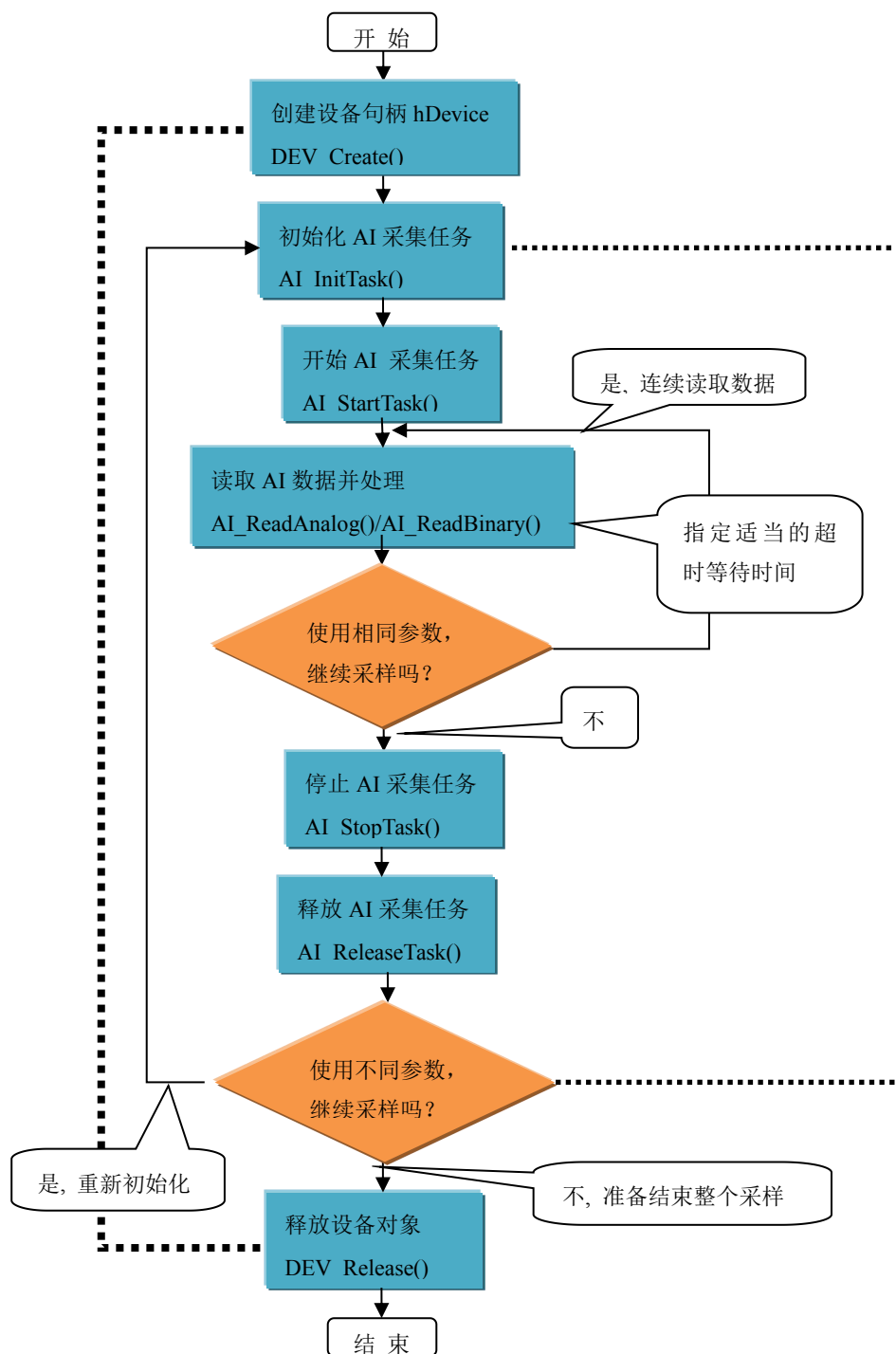


图 2-1-1 AI 连续采样流程图例

2.4 如何用有限采集方式取得 AD 数据

有限点采样模式，就是在一次开始后，AI 按照设定的采样速率，触发条件进行指定时间的或指定点数的连续采样，达到指定点数后设备会自动停止。且在采样结束后才可以读取到完整的数据片断。具体流程如图 2-1-2 所示：

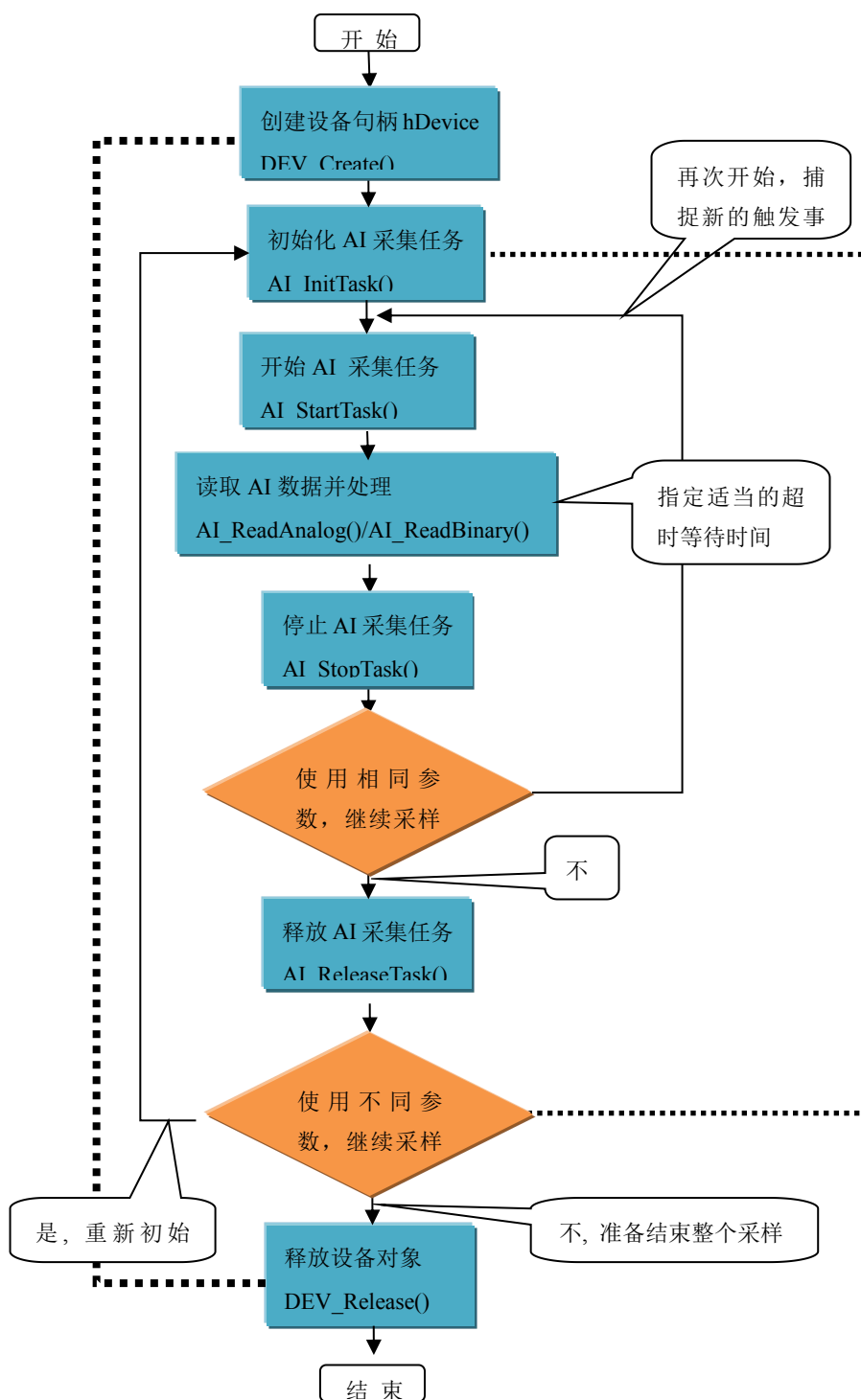


图 2-1-2 AI 有限点采样流程图例

2.5 哪些函数对您不是必须的

如果您使用上层用户函数访问设备，那么 [GetDeviceBar](#), [WriteRegisterByte](#), [WriteRegisterWord](#), [WriteRegisterULong](#), [ReadRegisterByte](#), [ReadRegisterWord](#), [ReadRegisterULong](#) 等函数您可完全不必理会，除非您是作为底层用户管理设备。而 [WritePortByte](#), [WritePortWord](#), [WritePortULong](#), [ReadPortByte](#), [ReadPortWord](#), [ReadPortULong](#) 则对 PCIe 用户来讲，可以说完全是辅助性，它们只是对我公司驱动程序的一种功能补充，对用户额外提供的，它们可以帮助您在 NT、Win2000 等操作系统中实现对您原有传统设备如 ISA 卡、串口卡、并口卡的访问，而没有这些函数，您可能在基于 Windows NT 架构的操作系统中无法继续使用您原有的老设备。

■ 3 ACTS 设备操作函数接口介绍

由于我公司的设备应用于各种不同的领域，有些用户可能根本不关心硬件设备的控制细节，只关心 AD 的首末通道、采样频率等，然后就能通过一两个简易的采集函数便能轻松得到所需要的 AD 数据。这方面的用户我们称之为上层用户。那么还有一部分用户不仅对硬件控制熟悉，而且由于应用对象的特殊要求，则要直接控制设备的每一个端口，这是一种复杂的工作，但又是必须的工作，我们则把这一群用户称之为底层用户。因此总的看来，上层用户要求简单、快捷，他们最希望在软件操作上所要面对的全是他们最关心的问题，比如在正式采集数据之前，只须用户调用一个简易的初始化函数（如 [InitDeviceAD](#)）告诉设备我要使用多少个通道，采样频率是多少赫兹等，然后便可以用 [ReadDeviceAD](#) 函数指定每次采集的点数，即可实现数据连续不间断采样。而关于设备的物理地址、端口分配及功能定义等复杂的硬件信息则与上层用户无任何关系。那么对于底层用户则不然。他们不仅要关心设备的物理地址，还要关心虚拟地址、端口寄存器的功能分配，甚至每个端口的 Bit 位都要了如指掌，看起来这是一项相当复杂、繁琐的工作。但是这些底层用户一旦使用我们提供的技术支持，则不仅可以让您不必熟悉 PCIe 总线复杂的控制协议，同是还可以省掉您许多繁琐的工作，比如您不用去了解 PCIe 的资源配置空间、PNP 即插即用管理，而只须 [GetDeviceBar](#) 函数便可以同时取得指定设备的地址。这个时候您便可以用这个地址，再根据硬件使用说明书中的各端口寄存器的功能说明，然后使用 [ReadRegisterULong](#) 和 [WriteRegisterULong](#) 对这些端口寄存器进行 32 位模式的读写操作，即可实现设备的所有控制。

综上所述，用户使用我公司提供的驱动程序软件包将极大的方便和满足您的各种需求。但为了您更省心，别忘了在您正式阅读下面的函数说明时，先明白自己是上层用户还是底层用户，因为在《[设备驱动接口函数总列表](#)》中的备注栏里明确注明了适用对象。

3.1 设备驱动接口函数总列表（每个函数省略了前缀“ACTS1200_”）

函数名	函数功能	备注
① 设备对象操作函数		
DEV_Greate	创建设备对象句柄	上层及底层用户
DEV_GetCount	取得同一种设备的总台数	上层及底层用户
DEV_GetSpeed	读取设备连接的 USB 端口速度	上层及底层用户
DEV_GetCurrentIdx	获得指定设备中的逻辑序号和物理序号	上层及底层用户
DEV_Release	释放设备对象(关键函数)	上层及底层用户
② AI 模拟量输入实现函数		
AI_GetDDR2Length	返回板载 DDR2 大小	上层用户
AI_InitTask	初始化 AI 采集任务	上层用户
AI_StartTask	启动 AI 采集任务	上层用户
AI_SendSoftTrig	发送软件触发事件	上层用户
AI_GetStatus	取得 AI 各种状态	上层用户
AI_WaitUntilTaskDone	采样任务结束前等待	上层用户
AI_ReadBinary	连续读取指定通道组采样数据	上层用户
AI_StopTask	停止 AI 采集任务	上层用户
AI_ReleaseTask	释放采集任务	上层用户

③ AD 硬件参数系统保存、读取函数		
AI_LoadParam	从 ACTS1200.ini 中加载 AI 参数	上层用户
AI_SaveParam	保存 AI 参数至 ACTS1200.ini	上层用户
AI_ResetParam	将当前 AI 参数复位至出厂值	上层用户

使用需知:

Visual C++:

要使用如下函数关键的问题是:

首先, 必须在您的源程序中包含如下语句:

```
#include "C:\Art\ACTS1200\INCLUDE\ACTS1200.H"
```

注: 以上语句采用默认路径和默认板号, 应根据您的板号和安装情况确定 ACTS1200.H 文件的正确路径, 当然也可以把此文件拷到您的源程序目录中。

另外, 要在 VB 环境中用子线程以实现高速、连续数据采集与存盘, 请务必使用 VB5.0 版本。当然如果您有 VB6.0 的最新版, 也可以实现子线程操作。

3.2 设备对象管理函数原型说明

◆ 创建设备对象函数 (逻辑号)

函数原型:

Visual C++:

```
HANDLE DEV_Create (U32 nDeviceIdx, BOOL bUsePhysIdx)
```

功能: 创建设备对象(Create device object), 并返回其设备对象句柄 hDevice。只有成功获取 hDevice, 用户才能顺利调用其它相关的接口函数以实现了对设备的控制。

参数:

nDeviceIdx 入口参数, 设备序号(Device Index)。设备序号有两种: 逻辑序号(Logical Index)和物理序号(Physical Index)。逻辑序号的定义是: 当向同一台计算机系统中加入若干张相同类型的 USB 卡时, 驱动程序自动以逻辑号来管理每张卡。比如某台计算机系统中插入该卡共四张, 则根据操作系统的加载顺序依次分配的逻辑号为 0、1、2、3。因为每个设备的逻辑号是不能事先由用户硬性决定的, 而是由操作系统加载设备时的顺序决定的。而设备物理号则由可以用户事先对硬件进行配置决定的号, 这个号是固定的, 跟插入 USB 的顺序没有关系。究竟使用哪一种设备号, 由参数 bUsePhysIdx 决定。当使用物理序号时, 其取值范围为[0, 255]。

bUserPhysIdx 入口参数, 是否使用物理序号, =FALSE(0): 不使用物理序号而使用逻辑序号; =TRUE(1): 使用物理序号。所有设备均支持逻辑号, 但不一定支持物理号, 本设备支持。

返回值: 如果执行成功, 则返回设备对象句柄; 如果没有成功, 则返回错误码 INVALID_HANDLE_VALUE。由于此函数已带容错处理, 即若出错, 它会自动弹出一个对话框告诉您出错的原因。您只需要对此函数的返回值作一个条件处理即可, 别的任何事情您都不必做。

相关函数: [DEV_Create\(\)](#) [DEV_GetCount\(\)](#) [DEV_GetCurrentIdx\(\)](#)
[DEV_Release\(\)](#)

Visual C++ 程序举例

:

```
HANDLE hDevice;    // 定义设备对象句柄
int DeviceLgcID = 0;
    hDevice = ACTS1200_DEV_Create(0, 0); // 创建设备对象,并取得设备对象句柄
    if(hDevice == INVALID_HANDLE_VALUE); // 判断设备对象句柄是否有效
    {
        return;    // 退出该函数
    }
    :
```

◆ 取得本计算机系统中 ACTS1200 设备的总数量

函数原型:

Visual C++:

U32 DEV_GetCount(void)

功能: 取得 ACTS1200 设备的数量。

参数: 无

返回值: 如果有设备存在, 则返回实际的设备数量, 若该设备不存在, 则返回 0 值, 此则有两种可能的情况存在: 其一, 设备根本不存在, 致使驱动程序无法安装加载。其二、设备存在, 但其驱动程序未正确安装。对于具体原因, 可以在操作系统的“设备管理器”中查看是否有该设备的信息项。

相关函数: [DEV_Create\(\)](#) [DEV_GetCount\(\)](#) [DEV_GetCurrentIdx\(\)](#)
 [DEV_Release\(\)](#)

◆ 取得该设备当前相应的逻辑序号和物理序号

函数原型:

Visual C++:

BOOL DEV_GetCurrentIdx (HANDLE hDevice,
 U32* pLgcIdx,
 U32* pPhysIdx)

功能: 取得指定设备相应 ID 号。

参数:

功能: 取得指定设备物理号和逻辑号(Get logical and physical index of the device)。

参数:

hDevice 入口参数, 设备对象句柄, 由 [DEV_Create\(\)](#) 函数创建, 该句柄指向要访问的设备。

pLgcIdx 出口参数, 取得设备的逻辑索引号(Logical Index), 取值范围为[0, 255]。如果=NULL 则表示忽略此参数。

pPhysIdx 出口参数, 取得设备的物理索引号(Physical Index), 取值范围为[0, 255], 具体值由 hDevice 指定的硬件决定。如果=NULL 则表示忽略此参数。

返回值: 如果初始化设备对象成功, 则返回 TRUE, 否则返回 FALSE, 用户可用 GetLastErrorEx 捕获当前错误码, 并加以分析。

相关函数: [DEV_Create\(\)](#) [DEV_GetCount\(\)](#) [DEV_GetCurrentIdx\(\)](#)
[DEV_Release\(\)](#)

◆ 释放设备对象所占的系统资源及设备对象

函数原型:

Visual C++:

`BOOL DEV_Release (HANDLE hDevice)`

功能: 释放设备对象所占用的系统资源及设备对象自身。

参数:

hDevice 设备对象句柄, 它应由 [CreateDevice](#) 创建。

返回值: 若成功, 则返回 TRUE, 否则返回 FALSE, 用户可以用 GetLastErrorEx 捕获错误码。

相关函数: [CreateDevice](#)

应注意的是, [CreateDevice](#) 必须和 [ReleaseDevice](#) 函数一一对应, 即当您执行了一次 [CreateDevice](#) 后, 再一次执行这些函数前, 必须执行一次 [ReleaseDevice](#) 函数, 以释放由 [CreateDevice](#) 占用的系统软硬件资源, 如 DMA 控制器、系统内存等。只有这样, 当您再次调用 [CreateDevice](#) 函数时, 那些软硬件资源才可被再次使用。

3.3 AI 模拟量输入实现函数原型说明

◆ 初始化 AI 采集任务

函数原型:

Visual C++:

`BOOL AI_InitTask (HANDLE hDevice, PACTS1200_AI_PARAM pAIParam, HANDLE* pSampEvent)`

功能: 初始化 AI 任务参数(Initialize task parameter for analog input), 为设备操作就绪有关状态, 如预置 AI 采样率、各通道模拟量采样范围等。但它并不开始 AI 设备, 若要开始 AI 设备, 须在成功调用此函数之后再调用 [AI_StartTask\(\)](#) 函数。

参数:

hDevice 入口参数, 设备对象句柄, 由 [DEV_Create\(\)](#) 函数创建, 该句柄指向要访问的设备。

pAIParam 入口参数, AI 工作参数结构体指针, 决定了 AI 工作时的各种状态及参数, 如采样率等。关于其具体定义及说明请参考《4 各种结构体描述》\《4.1 AI_PARAM (AI 工作参数结构)》。

pSampEvent 出口参数, 事件句柄, 该事件属性为自动发信号, 初始状态为不发信号, 它由任务自动创建。如果该参数=NULL, 则视为用户不需要驱动程序触发任何事件。

该事件句柄的作用是: 当任务中每通道有 AIParam.nSampsPerChan 个采样点的数据时则会触发此事件。用户可调用 WIN32 API 函数 WaitForSingleObject() 来跟踪该事件。当事件未发生时, 调用 WaitForSingleObject() 函数的采集线程会自动阻塞(等待)状态(此时调用线程不会消耗 CPU 时间), 当事件发生时, 则意味着任务中至少有 nSampsPerChan 个数据可读, 则采集线程立即进入运行状态,

并迅速调用 [AI_ReadAnalog\(\)](#) 或 [AI_ReadBinary\(\)](#) 循环读取任务中的数据，直至 nAvailSampsPerChan 小于 nReadSampsPerChan。

如果简单循环调用 [AI_GetStatus\(\)](#) 查询 AI 的各种状态以同步数据读操作，则会在等待中消耗许多 CPU 时间，而影响系统的整体性能。如果采用事件同步相结合的办法，则会节省大量的 CPU 时间。因为在调用 WaitForSingleObject() 时，若事件未被触发，则当前线程会进入阻塞(等待)状态，而不消耗 CPU 时间，而事件一旦触发，则当前线程立即进入运行状态。总之它可以在一定程度上提升系统的整体性能，让应用程序有更多的 CPU 时间去处理采样数据或其他任务。当然最简单的办法则是使用 [AI_ReadAnalog\(\)](#) 或 [AI_ReadBinary\(\)](#) 函数的超时机制，即利用 fTimeout 参数的合理设置来同步读入操作，而无须关注和跟踪 pSampEvent 事件。

返回值：如果初始化设备对象成功，则返回 TRUE，且 AD 便被启动。否则返回 FALSE，用户可用 GetLastErrorEx 捕获当前错误码，并加以分析。

相关函数： [DEV_Create\(\)](#) [AI_InitTask\(\)](#) [AI_StartTask\(\)](#)
[AI_SendSoftTrig\(\)](#) [AI_GetStatus\(\)](#) [AI_WaitUntilTaskDone\(\)](#)
[AI_ReadAnalog\(\)](#) [AI_ReadBinary\(\)](#) [AI_StopTask\(\)](#)
[AI_ReleaseTask\(\)](#)

◆ 启动 AI 采集任务

函数原型：

Visual C++:

BOOL AI_StartTask(HANDLE hDevice)

功能：开始 AI 采集(Start task for analog input)。必须在成功调用 [AI_InitTask\(\)](#) 函数后才能调用此函数，调用该函数后 AI 立即准备就绪，但 AI 实际是否进入采集记录过程，须依赖于触发事件的产生。

参数：

hDevice 入口参数，设备对象句柄，由 [DEV_Create\(\)](#) 函数创建，该句柄指向要访问的设备。

返回值：如果调用成功，则返回 TRUE，AI 立刻被开始， 否则返回 FALSE，可立即调用 WIN32 API 函数 GetLastError() 捕获错误码以确定具体原因。

相关函数： [DEV_Create\(\)](#) [AI_InitTask\(\)](#) [AI_StartTask\(\)](#)
[AI_SendSoftTrig\(\)](#) [AI_GetStatus\(\)](#) [AI_WaitUntilTaskDone\(\)](#)
[AI_ReadAnalog\(\)](#) [AI_ReadBinary\(\)](#) [AI_StopTask\(\)](#)
[AI_ReleaseTask\(\)](#)

◆ 发送软件触发事件

函数原型：

Visual C++:

BOOL AI_SendSoftTrig(HANDLE hDevice)

功能：发送软件触发事件(Send software trigger event for analog input)。设备进入等待触发状态，若用户需软件触发或需要随时手动给触发事件时，可调用此函数。该方式也叫软件触发或手动触发或内触发。

参数:

hDevice 入口参数, 设备对象句柄, 由 [DEV_Create\(\)](#) 函数创建, 该句柄指向要访问的设备。

返回值: 如果调用成功, 则返回 TRUE, 即 AI 立刻被触发采样一次, 否则返回 FALSE, 可立即调用 WIN32 API 函数 [GetLastError\(\)](#) 捕获错误码以确定具体原因

注释: 此函数也可用于单点读取和几个点的读取, 只需要将 `nReadSizeWords` 设置成 1 或相应值即可。其使用方法请参考《[高速大容量、连续不间断数据采集及存盘技术详解](#)》章节。

相关函数: [DEV_Create\(\)](#) [AI_InitTask\(\)](#) [AI_StartTask\(\)](#)
 [AI_SendSoftTrig\(\)](#) [AI_GetStatus\(\)](#) [AI_WaitUntilTaskDone\(\)](#)
 [AI_ReadAnalog\(\)](#) [AI_ReadBinary\(\)](#) [AI_StopTask\(\)](#)
 [AI_ReleaseTask\(\)](#)

◆ 取得 AI 各种状态

函数原型:

Visual C++:

`BOOL AI_GetStatus (HANDLE hDevice, PACTS1200_AI_STATUS pStatus )`

功能: 取得 AI 的各种状态(Get status for analog input)。一旦调用 [AI_StartTask\(\)](#) 函数后, 应立即调用此函数查询 AI 状态去同步采样数据的读操作。`nAvailSampsPerChan>0` 时, 表示采集任务中至少有 1 个数据段可供读取, 用户应立即调用 [AI_ReadBinary\(\)](#) 或 [AI_ReadAnalog\(\)](#) 函数循环读取若干可读段数据, 直到 `nAvailSampsPerChan>nReadSampsPerChan` 时可调用读数函数。如果在开始采集任务后, 设备迟迟不能被触发采样, 可以根据需要调用 [AI_SendSoftTrig\(\)](#) 函数以强制触发设备采样, 便可以很快得到有效的可读采样数据。

参数:

hDevice 入口参数, 设备对象句柄, 由 [DEV_Create\(\)](#) 函数创建, 该句柄指向要访问的设备。

pAIStatus 出口参数, 设备状态参数结构体, 它返回设备当前的各种状态, 如是否完成采样、是否已被触发等信息。关于具体状态信息请参考《[4 各种结构体描述](#)》\《[4.2 AI_STATUS \(AI 状态信息结构\)](#)》。

返回值: 如果成功获取 AI 状态, 则返回 TRUE, 否则返回 FALSE, 可立即调用 WIN32 API 函数 [GetLastError\(\)](#) 捕获错误码以确定具体原因

相关函数: [DEV_Create\(\)](#) [AI_InitTask\(\)](#) [AI_StartTask\(\)](#)
 [AI_SendSoftTrig\(\)](#) [AI_GetStatus\(\)](#) [AI_WaitUntilTaskDone\(\)](#)
 [AI_ReadBinary\(\)](#) [AI_StopTask\(\)](#) [AI_ReleaseTask\(\)](#)

◆ 采样任务结束前等待

函数原型:

Visual C++:

`BOOL AI_WaitUntilTaskDone (HANDLE hDevice, F64 fTimeout)`

功能: 在 AI 的采集任务结束前等待(Wait until task done for analog input)。一旦调用 [AI_StartTask\(\)](#) 函数后, 可以调用此函数等待采集任务结束。它通常用在有限点采样模式中。

参数:

hDevice 入口参数，设备对象句柄，由 [DEV_Create\(\)](#) 函数创建，该句柄指向要访问的设备。

fTimeout 入口参数，超时时间，单位：秒（S）。指定该次等待所用时间，比如设定为 10.0，即 10 秒钟的时间，如果在 10 秒内采集任务结束，则函数立即返回 TRUE，否则 10 秒钟后函数返回值 FALSE，如果采样速率极慢或触发事件长时间都不能达到的情况下，建议该超时时间应足够长；如果想禁止超时返回（即总是等到采集任务结束才返回）则赋值小于 0 即可，如-1.0；如果 fTimeout=0.0，则意味着该函数只是简单查询采集任务是否结束，如果采集任务结束了，则返回 TRUE，否则返回 FALSE。

返回值：如果采集任务结束，则返回 TRUE，否则返回 FALSE，可立即调用 WIN32 API 函数 GetLastError() 捕获错误码以确定具体原因

应注意的是，[InitDeviceAD](#) 必须和 [ReleaseDeviceAD](#) 函数一一对应，即当您执行了一次 [InitDeviceAD](#) 后，再一次执行这些函数前，必须执行一次 [ReleaseDeviceAD](#) 函数，以释放由 [InitDeviceAD](#) 占用的系统软硬件资源，如映射寄存器地址、系统内存等。只有这样，当您再次调用 [InitDeviceAD](#) 函数时，那些软硬件资源才可被再次使用。

相关函数：

DEV_Create()	AI_InitTask()	AI_StartTask()
AI_SendSoftTrig()	AI_GetStatus()	AI_WaitUntilTaskDone()
AI_ReadBinary()	AI_StopTask()	AI_ReleaseTask()

◆ 连续读取采样数据

函数原型：

Visual C++:

```
LONG AI_ReadBinary(HANDLE hDevice,
                   U16 nBinArray[],
                   U32 nReadSampsPerChan,
                   U32* pSampsPerChanRead,
                   U32* pAvailSampsPerChan,
                   F64 fTimeout);
```

功能：读取二进制原码采样数据（Read binary data from the task）。该函数不对采样结果进行物理量的换算，它是设备采样后的直接结果。二进制原码具有数据紧凑、数据量小、传输快、处理快、存盘也快的特点。

参数：

hDevice 入口参数，设备对象句柄，由 [DEV_Create\(\)](#) 函数创建，该句柄指向要访问的设备。

nBinArray 出口参数，用户缓冲区，用于接收所有采样通道二进制原码数据，值区间[-32768, 32767]。各个采样通道的数据点是依次交替排列的。它点空间不能小

nReadSampsPerChan*AIParam.nSampChanCount。

在有限点和连续采样模式中，它指定该次从设备的当前可读数据位置读取的数据点数（单位：点）。注意此参数的值如大于当前的可读数点 **nAvailSampsPerChan** 则会继续等待直到至少有 **nReadSampsPerChan** 个点可读后读函数才会返回。等待期间，如果所等时间超过 **fTimeout** 指定时间也会返回，并置超时错误码。

在连续采样过程中，如果要保持连续不丢点，此参数应尽可能接近于甚至等于当前的可读点数 (**nAvailSampsPerChan**)，但不能大于 **AIParam.nSampsPerChan**。当然此参数值也不能大于 **fAnlgArray** 的缓冲区长度，所以为避免出错，所开辟的缓冲区不能小于 **nReadSampsPerChan*AIParam.nSampChanCount**。

pSampsPerChanRead 出口参数，返回每通道实际读取的点数。在单点采样模式中，如果读取成功，返回的每通道已读取点数总是为 1。注意每次都要检查 pSampsPerChanRead 的返回值，如果返回值等于 0，则要慎重处理。

pAvailSampsPerChan 出口参数，返回该次读操作完成时的每通道还可读而未读的数据点数。它跟 AI_GetStatus()函数取得的状态信息 AIStatus.nAvailSampsPerChan 是同一个状态信息。返回可读点数的意义在于通过数据读操作直接提供给用户，避免再次调用 AI_GetStatus()函数，即在读数据函数返回时判断可读点数，若大于 nReadSampsPerChan，则可紧接着再次调用读数据函数，直到 pAvailSampsPerChan 返回值小于 nReadSampsPerChan。在单点采样模式中,它的返回值总是为 0。

fTimeout 入口参数，超时时间，单位：秒 (S)。指定等待写入额定点数的时间。比如设定为 10.0，如果在 10 秒内读取的点数达到 nReadSampsPerChan 时立即返回 TRUE，否则 10 秒钟后函数返回 FALSE，并通过 pAvailSampsPerChan 告之实际读入的点数，如果采样速率极慢或触发事件长时间都不能达到的情况下，建议该超时时间应足够长；如果想禁止超时返回（即等待请求数据点数完全从任务中读到才返回）则置为负数，如-1.0 即可。如果 fTimeout=0.0，则意味着该函数仅简单判断能否立即读取到请求的点数，如果不能，则不等待，立即返回 FALSE，否则返回 TRUE。

返回值：如果函数调用成功则返回 TRUE，否则返回 FALSE，可以调用 Win32 API 函数 GetLastError()以取得进一步的错误码信息 nErrorCode，其定义如下表。

相关函数：	DEV_Create()	AI_InitTask()	AI_StartTask()
	AI_SendSoftTrig()	AI_GetStatus()	AI_WaitUntilTaskDone()
	AI_ReadBinary()	AI_StopTask()	AI_ReleaseTask()

◆ 停止 AI 采集任务

函数原型：

Visual C++:

BOOL AI_StopTask (HANDLE hDevice)

功能：停止 AI 采样(Stop task for analog input)。必须在成功调用 [AI_StartTask\(\)](#)函数后才能调用此函数。该函数除了停止 AI 采集外不改变设备的其他状态。

参数：

hDevice 入口参数，设备对象句柄，由 [DEV_Create\(\)](#)函数创建，该句柄指向要访问的设备。

返回值：如果调用成功，则返回 TRUE，且 AI 立刻停止转换， 否则返回 FALSE，可立即调用 WIN32 API 函数 GetLastError()捕获错误码以确定具体原因。

相关函数：	DEV_Create()	AI_InitTask()	AI_StartTask()
	AI_SendSoftTrig()	AI_GetStatus()	AI_WaitUntilTaskDone()
	AI_ReadBinary()	AI_StopTask()	AI_ReleaseTask()

◆ 释放采集任务

函数原型：

Visual C++:

BOOL AI_ReleaseTask (HANDLE hDevice)

功能：释放 AI(Release task for analog input)。必须在重新调用 [AI_InitTask\(\)](#) 函数之前被调用一次，即该函数必须和 [AI_InitTask\(\)](#)成对出现。注意此函数在内部首先执行 [AI_StopTask\(\)](#)函数停止 AI 采集后，才释放被占用的 AI 资源。

函数原型:

Visual C++:

```
BOOL AI_LoadParam (HANDLE hDevice,
                   PACTS1200_AI_PARAM pAIParam)
```

功能: 负责从 Windows 系统中读取设备的硬件参数。

参数:

hDevice 设备对象句柄, 它应由 [CreateDevice](#) 创建。

pADPara 属于 ACTS1200_PARA_AD 的结构指针类型, 它负责返回 PCIe 硬件参数值, 关于结构指针类型 ACTS1200_PARA_AD 请参考 ACTS1200.h 或 ACTS1200.Bas 或 ACTS1200.Pas 函数原型定义文件, 也可参考本文《[硬件参数结构](#)》关于该结构的有关说明。

返回值: 若成功, 返回 TRUE, 否则返回 FALSE。

相关函数: [DEV_Greate](#) [AI_LoadParam](#) [AI_SaveParam](#)
[AI_ReleaseDevice](#)

◆ AD 采样参数复位至出厂默认值函数

函数原型:

Visual C++:

```
BOOL AI_ResetParam (HANDLE hDevice,
                   PACTS1200_AI_PARAM pAIParam)
```

功能: 将系统中原来的 AD 参数值复位至出厂时的默认值。以防用户不小心将各参数设置错误造成一时无法确定错误原因的后果。

参数:

hDevice 设备对象句柄, 它应由 [CreateDevice](#) 创建。

pADPara 设备硬件参数, 它负责在参数被复位后返回其复位后的值。关于 ACTS1200_PARA_AD 的详细介绍请参考 ACTS1200.h 或 ACTS1200.Bas 或 ACTS1200.Pas 函数原型定义文件, 也可参考本文《[硬件参数结构](#)》关于该结构的有关说明。

返回值: 若成功, 返回 TRUE, 否则返回 FALSE。

相关函数: [DEV_Greate](#) [AI_LoadParam](#) [AI_SaveParam](#)
[AI_ResetParam](#) [AI_ReleaseDevice](#)

■ 4 硬件参数结构

4.1 AI 工作参数结构体 (ACTS1200_AI_PARAM)

Visual C++:

```
typedef struct _ACTS1200_AI_PARAM
{
    U32 nSampChanCount;
    U32 nMLength;
    U32 nNLength;
    U32 nSampsPerChan;
    U32 nReserved0;
    U32 nReserved1;

    ACTS1200_AI_CH_PARAM CHParam[ACTS1200_AI_MAX_CHANNELS];

    U32 nPFISel;
    U32 nFreqDivision;
    U32 nSampleMode;
    U32 nReferenceClock;
    U32 nTimeBaseClock;
    U32 bMasterEn;
    U32 bClkOutEn;
    U32 bSyncClkOutEn;
    U32 bSyncTrigOutEn;
    U32 nClkOutSel;
    U32 nReserved2;

    U32 nTriggerMode;
    U32 nTrigSrcSel;
    U32 nTrigType;
    U32 nTriggerDir;
    F32 fTriggerLevel;
    U32 nTriggerSens;
    U32 nTrigCount;
    U32 bTrigOutEn;
    U32 nTrigOutPolarity;
    U32 nTrigOutWidth;
    U32 nReserved3;

    U32 nReserved4;
    U32 nReserved5;
    U32 nReserved6;
```

```
U32 nReserved7;
} ACTS1200_AI_PARAM, *PACTS1200_AI_PARAM;
```

4.2 AI_CH_PARAM(AI 通道参数结构体)

bChannelEn 通道使能,1 使能 0 禁止。

nSampleRange 采样范围选择。

常量名	常量值	功能定义
ACTS1200_AI_SAMP_RANGE_N5_P5V	0	±5V
ACTS1200_AI_SAMP_RANGE_N1_P1V	1	±1V

nCouplingType 耦合类型。

常量名	常量值	功能定义
ACTS1200_AI_COUPLING_DC	0	直流耦合
ACTS1200_AI_COUPLING_AC	1	交流耦合

nInputImped 输入阻抗(1M Ω , 75 Ω), (仅板卡 8912/8914 支持)

常量名	常量值	功能定义
ACTS1200_AI_IMP_1M	0	1M Ω
ACTS1200_AI_IMP_75	1	75 Ω

nPFISel PFI 功能选择, 详见下面常量定义(仅板卡 8582 8584 支持)

常量名	常量值	功能定义
ACTS1200_PFISEL_TRIG_OUT	0	触发输出
ACTS1200_PFISEL_TRIG_IN	1	触发输入

nSampleMode 采样模式。它的选项值如下表:

常量名	常量值	功能定义
ACTS1200_AI_SAMP_MODE_ONE_DEMAND	0	软件按需单点采样(此卡不支持)
ACTS1200_AI_SAMP_MODE_FINITE	1	有限点采样
ACTS1200_AI_SAMP_MODE_CONTINUOUS	2	连续采样(后触发和硬件延时触发支持连续采样)

nTrigSrcSel 触发源使用选项。

常量名	常量值	功能定义
ACTS1200_AI_TRIGSRC_SOFT	0	内触发, 即软件触发
ACTS1200_AI_TRIGSRC_DTR	1	外部数字触发(DTR)

ACTS1200_AI_TRIGSRC_TRIGGER	2	同步信号触发（用于多卡同步）
ACTS1200_AI_TRIGSRC_CH0	3	通道 0 触发
ACTS1200_AI_TRIGSRC_CH1	4	通道 1 触发
ACTS1200_AI_TRIGSRC_CH2	5	通道 2 触发
ACTS1200_AI_TRIGSRC_CH3	6	通道 3 触发

TrigLevelVolt 触发电平范围为 nTrigSrcSel 所选择通道的输入量程)。

nTriggerSens 触发灵敏窗单位 nS,步进为本卡最高采样率的采样周期;例如最高 80M,步进即为 12.5nS

nTrigCount 触发次数设置,默认为 1 次,此功能仅在后触发和硬件延时触发模式下有效,触发次数*有效读取点数*使能通道数*2<=内存大小(字节)

bTrigOutEn 触发输出使能 =1:使能;=0:禁止使能。

nTrigOutPolarity 触发输出极性选择。它的选项值如下表:

常量名	常量值	功能定义
ACTS1200_AI_TOP_NEGATIVE	0	负脉冲输出
ACTS1200_AI_TOP_POSITIVE	1	正脉冲输出

nTriggerDir 触发方向选择。它的选项值如下表:

常量名	常量值	功能定义
ACTS1200_AI_TRIGDIR_FALLING	0	下降沿
ACTS1200_AI_TRIGDIR_RISING	1	上升沿
ACTS1200_AI_TRIGDIR_CHANGE	2	变化

nReferenceClock 参考时钟选择。它的选项值如下表:

常量名	常量值	功能定义
ACTS1200_AI_RECLK_ONBOARD	0	板载时钟(默认)
ACTS1200_AI_RECLK_EXT_10M	1	外部 10M(CLK_IN)(板卡 8582 8584 不支持)
ACTS1200_AI_RECLK_MAIN_10M	2	主卡同步 10M

nTimeBaseClock 采样时基选择。它的选项值如下表:

常量名	常量值	功能定义
ACTS1200_AI_TBCLK_IN	0	内部时钟(默认)
ACTS1200_AI_RECLK_EXT	1	外部时钟(CLK_IN)(板卡 8582 8584 不支持)

nClkOutSel 时钟输出选择。它的选项值如下表:

常量名	常量值	功能定义
ACTS1200_AI_TRIGDIR_FALLING	0	下降沿
ACTS1200_AI_TRIGDIR_RISING	1	上升沿
ACTS1200_AI_TRIGDIR_CHANGE	2	变化

nTriggerMode 触发模式所使用的选项

常量名	常量值	功能定义
ACTS1200_AI_TRIGMODE_MIDL	0	中间触发:M 为触发前采集点数,N 为触发后采集点数
ACTS1200_AI_TRIGMODE_POST	1	后触发:M 无效,N 为触发后采集点数
ACTS1200_AI_TRIGMODE_PRE	2	预触发:M 无效,N 为触发前采集点数
ACTS1200_AI_TRIGMODE_DELAY	3	硬件延时触发:M 为触发后延时点数,N 为延时后采集点数

nTrigOutWidth 触发脉冲输出宽度单位 nS, [50, 50000] 步进 50nS。

相关函数: [CreateDevice](#) [LoadParaAD](#) [SaveParaAD](#)
[ReleaseDevice](#)

4.3 AI 状态参数结构 (ACTS1200_AI_STATUS)

Visual C++:

```
typedef struct _ACTS1200_AI_STATUS
```

```
{
```

```
    U32 bTaskDone;           // 采样任务完成标志, =TRUE:表示采样任务完成, =FALSE:
```

表示采样未完成, 正在进行中

```
    U32 bTriggered;         // 触发标志, =TRUE:表示已被触发, =FALSE:表示未被触发
```

(即正等待触发)

```
    U32 nSampTaskState;     // 采样任务状态, =1:正常, 其它值表示有异常情况
```

```
    U32 nAvailSampsPerChan; // 每通道有效点数, 只有它大于当前指定读数长度时才能调用 AI_ReadAnalog()立即读取指定长度的采样数据
```

```
    U32 nMaxAvailSampsPerChan; // 自 AI_StartTask()后每通道出现过的最大有效点数, 状态值范围[0, nBufSampsPerChan],它是为监测采集软件性能而提供, 如果此值越趋近于 1, 则表示意味着性能越高, 越不易出现溢出丢点的可能
```

```
    U32 nBufSampsPerChan;   // 每通道缓冲区大小(采样点数)。
```

```
    I64 nSampsPerChanAcquired; // 每通道已采样点数(自启动 AI_StartTask()之后所采样的点数), 这个只是给用户的统计数据
```

```
    U32 nHardOverflowCnt;    // 硬件溢出计数(在不溢出情况下恒等于 0)
```

```
    U32 nSoftOverflowCnt;    // 软件溢出计数(在不溢出情况下恒等于 0)
```

```
    U32 nInitTaskCnt;        // 初始化采样任务的次数(即调用 AI_InitTask()的次数)
```

```
    U32 nReleaseTaskCnt;     // 释放采样任务的次数(即调用 AI_ReleaseTask()的次数)
```

```
    U32 nStartTaskCnt;       // 启动采样任务的次数(即调用 AI_StartTask()的次数)
```

```
    U32 nStopTaskCnt;        // 停止采样任务的次数(即调用 AI_StopTask()的次数)
```

U32 nTransRate; // 传输速率, 即每秒传输点数(sps), 作为 USB 及应用软件传输性能的监测信息

```
U32 nReserved0; // 保留字段(暂未定义)
U32 nReserved1; // 保留字段(暂未定义)
U32 nReserved2; // 保留字段(暂未定义)
U32 nReserved3; // 保留字段(暂未定义)
U32 nReserved4; // 保留字段(暂未定义)
} ACTS1200_AI_STATUS, *PACTS1200_AI_STATUS;
```

此结构体主要用于查询 AI 的各种状态, 以便同步各种数据采集和处理过程。

相关函数: [CreateDevice](#) [ReleaseDevice](#)

4.4 AI 主要信息结构 (ACTS1200_AI_MAIN_INFO)

Visual C++:

```
typedef struct _ACTS1200_AI_MAIN_INFO
```

```
{
```

```
U32 nDeviceType; // 总线类型\设备类型 0x30638514 30638512...3063(USB)
```

```
U32 nChannelCount; // AI 通道数量
```

```
U32 nDepthOfMemory; // AI 板载存储器深度(MB)
```

U32 nSampResolution; // AI 采样分辨率(如=8 表示 8Bit; =12 表示 12Bit; =14 表示 14Bit; =16 表示 16Bit)

```
U32 nSampCodeCount; // AI 采样编码数量(如 256, 4096, 16384, 65536)
```

U32 nTrigLvlResolution; // 触发电平分辨率(如=8 表示 8Bit; =12 表示 12Bit; =16 表示 16Bit)

```
U32 nTrigLvlCodeCount; // 触发电平编码数量(如 256, 4096)
```

```
U32 nTimerBase; // 时钟基准(即板载晶振频率)单位:Hz
```

```
U32 nMaxRate; // 最大采样速率(sps)
```

```
U32 nMinFreqDivision; // 最小分频数
```

```
U32 nSupportImped; // 是否支持输入阻抗(0:不支持 1:支持)
```

```
U32 nSupportPFI; // 是否支持 PFI 触发输入输出选择(0:不支持 1:支持)
```

```
U32 nSupportExtClk; // 是否支持外时钟(0:不支持 1:支持)
```

```
U32 nSupportClkOut; // 是否支持时钟输出(0:不支持 1:支持)
```

```
U32 nLocalVersion; // Local 端版本号
```

U32 nProductSeries; // 产品序列(3200H:多功能系列;2800H:低速同步系列;8500H:高速同步系列 8900H:高速数字化仪)

```
U32 nEPSize; // 载 E2P 存储字节深度(Byte)
```

```
U32 nSampRangeCount; // AI 采样范围挡位数量
```

```
U32 nSampGainCount; // AI 采样增益挡位数量
```

```
U32 nCouplingCount; // AI 耦合挡位数量
```

```
U32 nImpedanceCount; // AI 阻抗的挡位数量
```



```

    U32 nReserved0;          // 保留字段(暂未定义)
    U32 nReserved1;          // 保留字段(暂未定义)
    U32 nReserved2;          // 保留字段(暂未定义)
    U32 nReserved3;          // 保留字段(暂未定义)
} ACTS1200_AI_MAIN_INFO, *PACTS1200_AI_MAIN_INFO;

```

4.5 AI 采样范围信息结构体(ACTS1200_AI_SAMP_RANGE_INFO)

Visual C++:

```

typedef struct _ACTS1200_AI_SAMP_RANGE_INFO
{
    U32 nSampleRange;        // 当前采样范围挡位号
    U32 nReserved0;          // 保留字段(暂未定义)
    F64 fMaxVolt;            // 采样范围的最大电压值, 单位: 伏(V)
    F64 fMinVolt;            // 采样范围的最小电压值, 单位: 伏(V)
    F64 fAmplitude;          // 采样范围幅度, 单位: 伏(V)
    F64 fHalfOfAmp;          // 采样范围幅度的二分之一, 单位: 伏(V)
    F64 fCodeWidth;          // 编码宽度, 单位: 伏(V), 即每个单位码值所分配的电压值
    F64 fOffsetVolt;         // 偏移电压, 单位: 伏(V), 一般用于零偏校准
    F64 fOffsetCode;         // 偏移码值, 一般用于零偏校准, 它代表的电压值等价于
fOffsetVolt
    char strDesc[16];        // 采样范围字符描述, 如“±10V”, “0-10V”等
    U32 nPolarity;           // 采样范围的极性(0=双极性 BiPolar, 1=单极性 UniPolar)

    U32 nReserved1;          // 保留字段(暂未定义)
    U32 nReserved2;          // 保留字段(暂未定义)
    U32 nReserved3;          // 保留字段(暂未定义)
} ACTS1200_AI_SAMP_RANGE_INFO, *PACTS1200_AI_SAMP_RANGE_INFO;

```

4.6 AI 速率信息结构体(ACTS1200_AI_SAMP_RATE_INFO)

Visual C++:

```

typedef struct _ACTS1200_AI_SAMP_RATE_INFO
{
    F64 fMaxRate;            // 单通道最大采样速率(sps), 多通道时各通道平分最大采样率
    F64 fMinRate;            // 单通道最小采样速率(sps), 多通道时各通道平分最小采样率
    F64 fTimerBase;          // 时钟基准(即板载晶振频率) 单位: Hz
    U32 nDivideMode;         // 分频模式, 0=整数分频(INTDIV), 1=DDS 分频(DDSDIV)
    U32 nRateType;           // 速率类型, 指 fMaxRate 和 fMinRate 的类型, =0: 表示为所有采样
通道的总速率, =1: 表示为每个采样通道的速率

    U32 nReserved0;          // 保留字段(暂未定义)

```

```
U32 nReserved1;          // 保留字段(暂未定义)
} ACTS1200_AI_SAMP_RATE_INFO, *PACTS1200_AI_SAMP_RATE_INFO;
```

5 数据格式转换与排列规则

5.1 AI 原码 LSB 数据转换成电压值的换算方法

首先应根据设备实际位数屏蔽掉不用的高位，然后依其所选量程，按照下表公式进行换算即可。这里只以缓冲区 ADBuffer[] 中的第 1 个点 ADBuffer[0] 为例。

USB8914

量程	计算机语言换算公式（ANSIC语法）	Volt取值范围（mV）
±1000mV	$Volt = (2000.00/16384) * ((ADBuffer[0] \wedge 0x8000) \& 0x3FFF) - 1000$	[-1000, +999.51]
±5000mV	$Volt = (10000.00/16384) * ((ADBuffer[0] \wedge 0x8000) \& 0x3FFF) - 5000$	[-5000, +4997.56]

下面举例说明各种语言的换算过程（以±5V 量程为例）

Visual C++:

```
Lsb= ADBuffer[0]&0x3FFF;
Volt=(10000.00/16384)*Lsb-5000.00;
USB8912
```

量程	计算机语言换算公式（ANSIC语法）	Volt取值范围（mV）
±1000mV	$Volt = (2000.00/4096) * ((ADBuffer[0] \wedge 0x8000) \& 0xFFF) - 1000$	[-1000, +999.51]
±5000mV	$Volt = (10000.00/4096) * ((ADBuffer[0] \wedge 0x8000) \& 0xFFF) - 5000$	[-5000, +4997.56]

下面举例说明各种语言的换算过程（以±5V 量程为例）

Visual C++:

```
Lsb= ADBuffer[0]&0xFFF;
Volt=(10000.00/4096)*Lsb-5000.00;
```

注：USB 8582 8584 8502 8504 8512 8514 的换算方法同上

5.2 AI 采集函数的 ADBuffer 缓冲区中的数据排放规则

数据缓冲区索引号	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	...
通道号	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	...

■ 6 上层用户函数接口应用实例

6.1 简易程序演示说明

怎样使用 [ReadBinary](#) 函数直接取得 AI 数据

Visual C++:

其详细应用实例及正确代码请参考 Visual C++测试与演示系统，您先点击 Windows 系统的[\[开始\]](#)菜单，再按下列顺序点击，即可打开基于 VC 的 Sys 工程。

[\[程序\]](#) | [\[阿尔泰测控演示系统\]](#) | [\[ACTS1200 同步采集 4 路 AD\]](#) | [\[Microsoft Visual C++\]](#) | [\[简易代码演示\]](#) | [\[AI 采集\]](#)

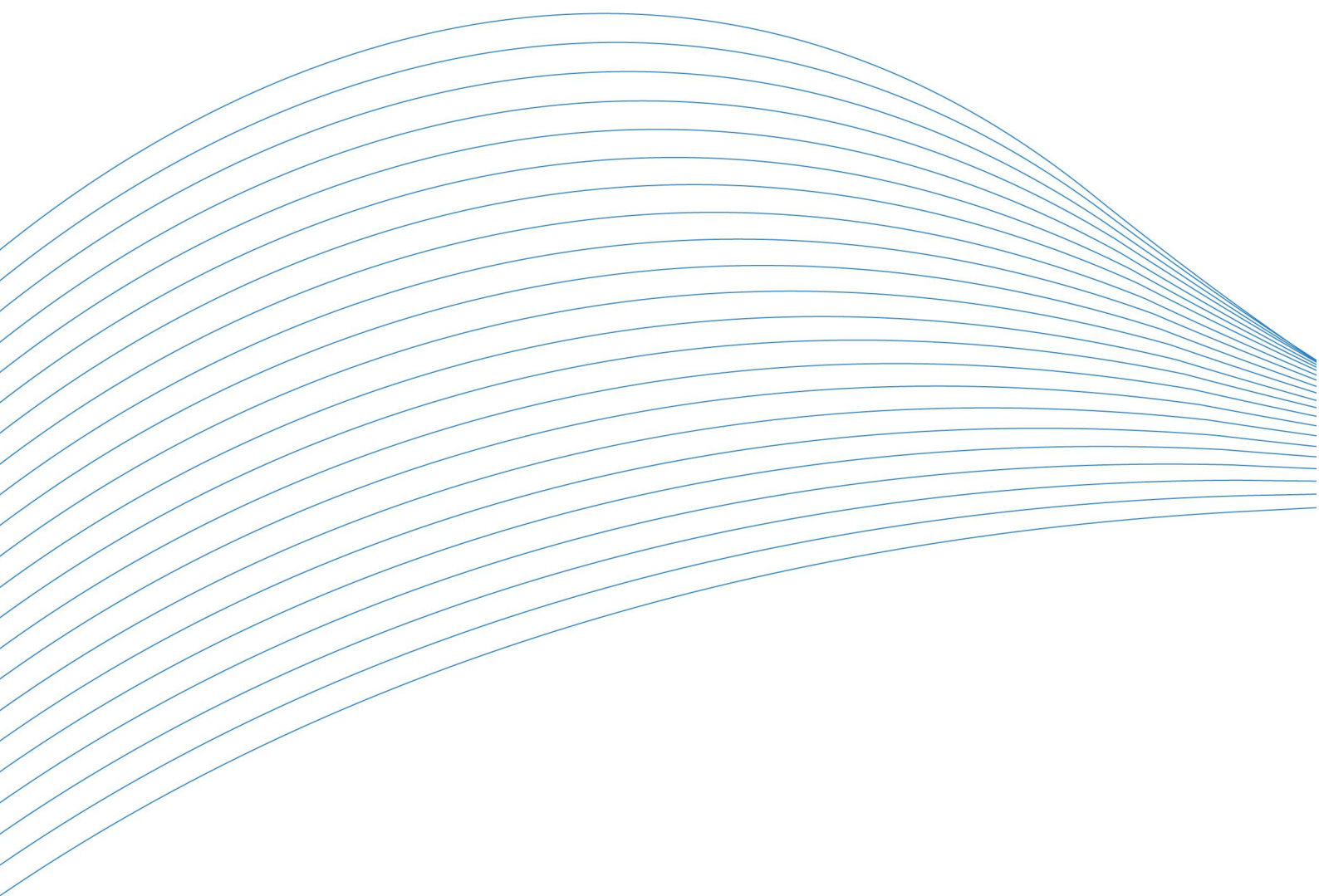
6.2 高级程序演示说明

高级程序演示了本设备的所有功能，您先点击 Windows 系统的[\[开始\]](#)菜单，再按下列顺序点击，即可打开基于 VC 的 Sys 工程(主要参考 ACTS1200.h 和 ADDoc.cpp)。

[\[程序\]](#) | [\[阿尔泰测控演示系统\]](#) | [\[ACTS1200 同步采集 4 路 AD\]](#) | [\[Microsoft Visual C++\]](#) | [\[高级代码演示\]](#)

其默认存放路径为：系统盘\ART\ACTS1200\SAMPLES\VC\ADVANCED

其他语言的演示可以用上面类似的方法找到。



北京阿尔泰科技发展有限公司

服务热线：400-860-3335

邮编：100086

传真：010-62901157